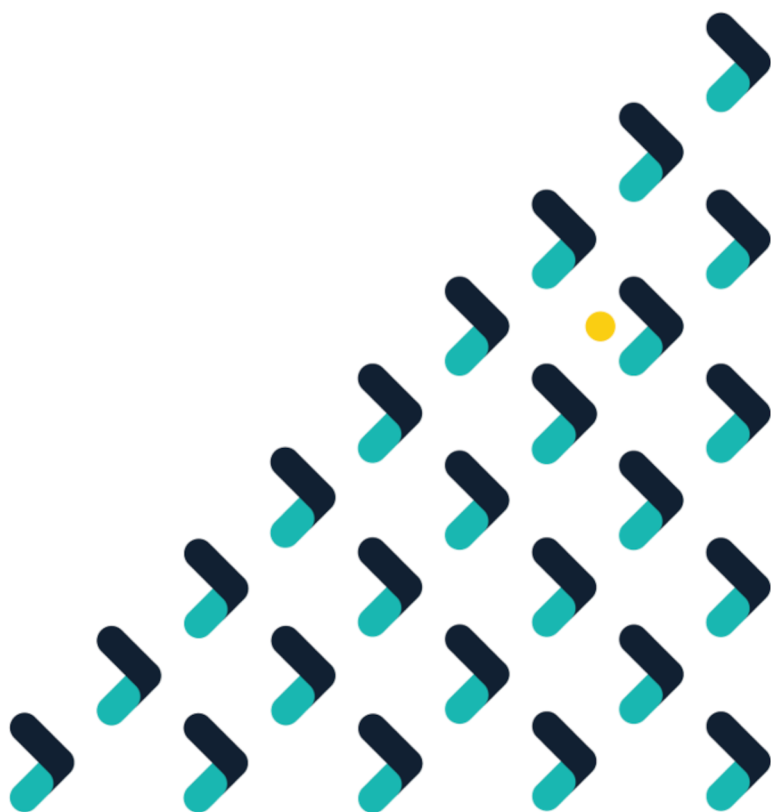




FO/FO2 Richtlinien

Forterro Deutschland Abas GmbH



Inhaltsverzeichnis

Sprache innerhalb der FO2 Programmierung	1
Programme	2
Ablageort der individuellen Programme	2
Struktur der Programme	2
Was ist eine Hülse?	2
Regeln für Hülse	3
Benennung von Programmen	4
Benennung EFOPs	4
Benennung Unterprogramme	5
Benennung manuell ausgeführte Programme	5
Benennung Cronjobs	6
Benennung Infosystemprogramme	6
Benennung INI-Dateien	7
Benennung von Variablen in FO/FO2 (.type/.var)	7
Events	9
Programmkopf und Interpreterzeile	11
Programmkopf	11
Interpreterzeile im FO/FO2	11
.set trans - / .set trans +	11
Sprachunabhängige Programmierung	13
Eventkommandos und Eventtaste sprachneutral	13
G evtkommd (bzw. G avtkommd)	13
G evttaste (bzw. G avttaste) beim Maskenende	13
Art des Ereignisses sprachneutral (G evtartengl)	14
Datenbankangabe in Englisch (z.B. .select/.load)	14
Möglichkeiten zur Ermittlung des englischen Datenbanknamens	14
Kommandosyntax	15
Dateinamen, Dateiaktionen, Tippkommandos, Sonstige (Laderkommandos) (s3sys/KOMLIST)	15
Aufzählungen	16
Einheiten	17
Übersetzungen Wörterbuch	18
Batch-User (für EDP-Aufrufe)	19
Dokumentation der Anpassungen	20
Dokumentation von Abschnitten (FO) bzw. Prozeduren (FO2)	20
Dokumentation im Quellcode	20
Sprachenauswahl für die Programmdokumentation	20
Bestehende Anpassungen erweitern	21
FO-Kommandos – Besonderheiten	22
Allgemeines (Puffer, keine Abkürzung)	22
Formatierung	22
Temporärdateien	22
Protokollierung	22
Fehlerdateien	22

Kommandos für Datenbankzugriffe: .select bzw. .add bzw. .load	23
Erklärung zur Lesesperre bei .select bzw. .load ohne Sperroption	23
Angabe Datenbankname beim Selektieren	23
Kommando zum Zurückschreiben bzw. Erstellen von Datensätzen: .rewrite	24
Unterprogramme	24
Transaktionen	24
Programmausgaben bzw. Interaktion (.box , .menu , .lesen , etc.)	25
Schutz von Feldern: .protection	25
Richtlinien – speziell FO	26
Programmkopf	26
FO-Header	26
Bestandteile	27
Sprungmarken	27
Richtlinien – speziell FO2	28
Programmkopf	28
FO2-Header	28
Bestandteile	29
Main Methode	29
Prozeduren	29
Individualisierung von Infosystemen (IS)	30
Druckindividualisierung	31

Sprache innerhalb der FO2 Programmierung

FO- und FO2-Programme sind auf Englisch zu programmieren, d.h. Kommandos und deren Optionen werden auf Englisch angegeben.

Es wird jedoch grundsätzlich mit deutschen (Datenbank-)Variablennamen programmiert, da dies im überwiegenden Teil der Installationen und im Standard so vorgegeben ist und eine Umstellung auf "englvar" für alle Abas-Nutzer ein starker Einschnitt wäre und die Geschwindigkeit der Programmierung stark verlangsamen würde. Zudem kann diese in einem Unterprogramm getätigte Einstellung im aufrufenden Hauptprogramm nicht wieder rückgängig gemacht werden, was zu schwerwiegenden Problemen führen kann.

Aus diesen Gründen ist die zu nutzende Interpreter-Zeile auch fest vorgegeben: *(siehe Programmkopf)*

Ausnahmen gelten bei Änderungen in bestehenden Installationen, hier sollte sich an die dort vorhandenen Gegebenheiten angepasst werden.

Programme

Ablageort der individuellen Programme

Anpassungen von FO-Programmen erfolgen ausschließlich in **ow**-Verzeichnissen. Diese sind upgradesicher und stellen dem Namen des jeweiligen Standardverzeichnisses das Präfix "ow" vor.

- Für individuelle Programme, welche nicht Bestandteil von individualisierten Infosystemen sind, wird "owfo" verwendet.
- Für individuelle Infosysteme wird "owis" verwendet.
- Für Cronjobs wird "owcron" verwendet.
- Für die Individualisierung von Standard Infosystemen werden weiterhin die individuellen Arbeitsbereiche (owev, owst, owvk, owek, etc.) verwendet.
- Protokolldateien werden in "owlog" gespeichert.
- Fehlerdateien werden in "owerror" (und Unterverzeichnissen) gespeichert.

Es werden keine neuen Programme im Mandantenverzeichnis oder in Standardverzeichnissen abgelegt.

Ausnahme: I-Files sind trotz Speicherung in Standardverzeichnissen upgradesicher. Bei Unsicherheit kann im Verzeichnis s3/refl geprüft werden, ob es sich bei dem betroffenen Programm um ein I-File handelt, oder ob es individualisiert werden muss.

Hinweise zu den Arbeitsbereichen:

- Alle Benutzer verwenden die identische Reihenfolge von Arbeitsbereichen (in denen u.a. die ow-Verzeichnisse enthalten sind).
- Von der Möglichkeit, FOPs ohne Arbeitsbereich aufzurufen und das entsprechende FOP automatisch gemäß Einordnung der ow-Verzeichnisse ermitteln zu lassen, machen wir keinen Gebrauch, da dies Fehlersuche und Debugging unnötig erschwert und Fehlerpotential bei zukünftigen Erweiterungen mit sich bringt.
- Falls aus Berechtigungsgründen Benutzer FOPs aus bestimmten Arbeitsbereichen nicht ausführen können sollen, muss eine Kopie der Arbeitsbereichsliste erstellt und die betroffenen Arbeitsbereiche entfernt werden. Die Reihenfolge der restlichen Arbeitsbereiche muss identisch bleiben.
- Damit individualisierte Standardinfosysteme mit gleichen Suchwort richtig aufgerufen werden, muss das ow-Verzeichnis vor dem Standard-Verzeichnis stehen. (z.B. Aufruf von (Infosystem) ANO: gibt es ein vk/ANO und ein owvk/ANO, dann muss im Arbeitsbereich owvk vor vk stehen. Damit würde das Kommando das individuelle Infosystem owvk/ANO anzeigen)

Struktur der Programme

Was ist eine Hülse?

In sog. Hülsen-FOPs, insbesondere für Maskenevents, werden in der Regel Unterprogramme eingebunden. Es ist stets darauf zu achten, dass der Kontext (Positionierung auf Kopf oder eine bestimmte Tabellenzeile) stets auf den Ausgangszustand bei Start des Unterprogramms zurückgesetzt wird, um andere Anpassungen nicht unbrauchbar zu machen.

Die Hülse wertet somit Maskenereignisse aus und entscheidet, welches Sub-FOP aufgerufen wird. Die Hülse selbst enthält keine Programmlogik.

Regeln für Hülsen

1. Es gibt 2 Lösungen für die Anwendung von Hülsen
 1. Sammlung aller Events in einem Sammler-EFOP
 2. Jedes Event erhält ein separates EFOP

Die Entscheidung für oder gegen einen der Ansätze muss situativ und nach Abschätzung durch den Umsetzenden erfolgen. Ziel ist es, eine gewisse Übersichtlichkeit zu schaffen, so dass nicht zu viel Quellcode in einem FOP enthalten ist, aber auch nicht unnötig viele einzelne FOPs erstellt werden, welche jeweils nur wenige Zeilen Code beinhalten.

Ereignis	Lösung a – Name und fop.txt (Eine Hülse pro Maske)	Lösung b – Name und fop.txt (Eine Hülse pro Maske und Event)
Maskenereignisse	0 * maskein * * K [C] owfo/S0000.F02	0 * maskein * * K [C] owfo/S0000.SE.F02
Feld- und Buttonereignisse	0 * felddaus * such K [C] owfo/S0000.F02	0 * felddaus * such K [C] owfo/S0000.KUNDE.FX.F02

Hinweis Lösung a: Hier muss pro Event, über die Variable G|evtkommd ein Bereich definiert werden. Der Nachteil in der Lösung ist, dass bei Masken, mit enorm vielen Ereignissen die Performance abnimmt, da das Programm immer "durchgelesen" werden muss bis es zum richtigen Event kommt.

Beispiel Hülse:

```
...
def main(){
  .input EVTCMD.CONST
  ..Mode
  switch (G|evtkommd) {
    case ('U|evtcmdNew') {
      ..Aufruf SUB FOP
    }
    case ('U|evtcmdEdit') {
      ..Aufruf SUB FOP
    }
    case ('U|evtcmdrelease') {
      ..Aufruf SUB FOP
    }
  }
}
```

Generell gilt:

1. Die Hülse übernimmt **ausschließlich Steuerungslogik**.
2. Berechnungen, Prüfungen, Datenmanipulationen gehören ausschließlich in Sub-FOPs.
3. Subfops müssen sauber dokumentiert werden, Mindestanforderung hierbei ist: **abas-[1.Buchstabe Vorname][Nachname], [Tagesdatum], [Arbeitspaketnummer] - [Titel des Arbeitspakets]**

4. In beiden Lösungen wird der Eintrag in der fop.txt immer an das jeweilige Event und die jeweilige Variable angebunden.
5. EFOPs sind bezüglich Eventkommandos (neu, ändern, freigeben, ...) mit "*" in der fop.txt anzubinden.
6. Die individuellen EFOPs sind in der fop.txt immer hinter den Standard-EFOPs anzubinden und zu kommentieren.
7. Standard-EFOPs in der fop.txt dürfen nicht angepasst werden.
8. Beim Einbinden von FOPs in die fop.txt ist zwingend auf die Angabe "[C]" zu achten, um sicherzustellen, dass ggf. weitere definierte EFOPs ebenfalls laufen, da der Default-Wert "[S]" ist.
9. Es darf **keine dedizierte Logik zum Durchlaufen von Tabellen** in der Hülse geben.
10. Die Hülse wird in der identischen Sprache wie die restlichen Programme geschrieben.

Sub-FOP's zur Sammlung bestimmter Funktionen – in gewissen Sub-FOP's kann man Funktionen wiederverwenden und bündeln:

- Vorbelegung bei Neuanlage oder Erstellung eines Folgevorschlags
- Vorbelegung von Skip-Feldern beim Maskeneintritt
- Pflichtfeldprüfungen bei der Maskenprüfung

Ausnahmen, sind nach Abstimmung mit Lead-Dev oder Teamlead Development insb. bei drohenden Performance-Problemen, möglich.

Benennung von Programmen

Der Dateiname von Programmen darf ausschließlich aus Großbuchstaben (ohne Umlaute), Ziffern und Punkten bestehen und muss mit einem Großbuchstaben beginnen. Es dürfen keine Umlaute benutzt werden.

Bei der Benennung von Programmen unterscheiden wir die folgenden Kategorien:

Benennung EFOPs

Ein EFOP (ereignisgesteuertes FOP) enthält die Bezeichnung nach dem folgenden Aufbau:

SXXXX.[Feldname.][Event].[FO2]

S	Konstante für Screen (Maske)
XXXX	4-stellige Maskennummer mit führenden Nullen
Feldname	Name des Feldes, mit dem das EFOP verbunden ist (optional, nur bei Feldereignissen)
Event	Abkürzung des Ereignisses, siehe Events
.FO2	Nur für FO2-Programme erforderlich

Beispiele:

- Maske Artikel, Ereignis Maskeneintritt → **S0002.SE.F02**
- Maske Auftrag, Feldaustritt Kunde → **S0032.KUNDE.FX.F02**

Benennung Unterprogramme

SUB.SXXXX.[Feldname.][Event].[Beschreibung].[FO2]

SUB	signalisiert, dass es sich um ein Unterprogramm handelt
S	Konstante für Screen (Maske)
XXXX	4-Stellige Maskennummer mit führenden Nullen. Sollte dieses Unterprogramm in mehreren Masken angebunden werden, dann wird statt der Nummer XXXX verwendet.
Feldname	Name des Feldes, mit dem das Unterprogramm verbunden ist (optional). Sollte dieses Unterprogramm in mehreren Feldern angebunden werden, dann wird statt des Feldnamens XX verwendet.
Event	Abkürzung des Ereignisses. Sollte dieses Unterprogramm an mehreren Events angebunden werden, dann wird statt des Events XX verwendet.
Beschreibung	Eine kurze Beschreibung, welche Funktion das Programm erfüllt.
.FO2	Nur für FO2-Programme erforderlich

Beispiele:

- Maske Auftrag, Diverse Feldaustritt- und Button-Ereignisse zum Thema Preisberechnung →
SUB.S0032.XX.XX.PRICECALC.FO2
- Diverse Verkaufsmasken, Feldaustritt Artikel, Ausfüllen von y-Feldern →
SUB.SXXXX.ARTIKEL.FX.FILLFIELDS.FO2

Benennung manuell ausgeführte Programme

FOP.[Beschreibung].[FO2]

FOP	Konstante zur Kennzeichnung von nicht ereignisgesteuerten FOPs
Beschreibung	Kurze Funktionsbeschreibung
.FO2	Nur für FO2-Programme erforderlich

Beispiele:

- Programm zum manuellen Datenexport von Buchungen für eine Schnittstelle →
FOP.EXPORTBUCHUNGEN.FO2

Hinweis: Grundsätzlich sind lediglich manuell aufzurufende FOP schwer wiederaufzufinden (da bspw. nur eingetragen in einem Arbeitsgruppenmenü). Daher sollten auch für manuell zu startende Programme Auslöser in Masken geschaffen werden. Es bietet sich immer an, ein neues Infosystem zu erstellen und

dessen Möglichkeiten zu nutzen, um in diesem Zusammenhang einen Mehrwert zu schaffen hinsichtlich Ausgabe von Informationen.

Benennung Cronjobs

Hierbei handelt es sich um zeitgesteuert ausgeführte Programme, welche in der Crontab eines entsprechenden Linux-Benutzers eingetragen werden.

CRON.[Beschreibung].[FO2]

CRON	Konstante zur Kennzeichnung von Cronjobs
Beschreibung	Kurze Funktionsbeschreibung
.FO2	Nur für FO2-Programme erforderlich

Beispiele:

- Cronjob zur täglichen Nachbewertung → **CRON.NACHBEWERTEN.FO2**

Grundsätzliche Strategien zur Einbindung von Cronjobs siehe (*Grundlagen: Cronjobs*).

Benennung Infosystemprogramme

Generell wird für neu erstellte Infosysteme nach Möglichkeit nur ein Programm für alle Events erzeugt. Unterprogramme dürfen aber, im Rahmen der Übersichtlichkeit, weiterhin verwendet werden, um bestimmte Unterfunktionen mit gewisser Komplexität abzubilden.

Der Name des Unterprogramms zum Infosystem beginnt mit dem Namen des Infosystems. Danach je Funktion sinnvoller Name (Beispiel: Infosystem ABC: Name des Unterprogramms **ABC.FUNKTIONXYZ.FO2**)

[INFOSYS].[Feldname].[Event].[FO2]

INFOSYS	Suchwort des Infosystems
Feldname	Name des Feldes, mit dem das EFOP verbunden ist (optional)
Event	Abkürzung des Ereignisses. In der Regel wird hier das Kürzel .EV verwendet, da das Programm an mehrere Events angebunden wird. In Ausnahmefällen gelten die üblichen Kürzel.
.FO2	Nur für FO2-Programme erforderlich

Bei Infosystemprogrammen ist zudem folgendes zwingend zu beachten:

- Unterprogramme müssen im Programmkopf mit einem **include** **[Arbeitsbereich] / [Programmname]** angegeben werden, damit sie beim Exportieren des Infosystems berücksichtigt und in das tgz-Archiv aufgenommen werden.

Beispiel:

```

..!interpreter english declaration noabbrev
..*****
..  Author           : [Author]
..  Date of Creation  : ${date}
..  Name             : ${filename}
..  Work package      :
..  Function          :
..  Modification      : [Author] [Date] [Changes]
..                   : [Author] [Date] [Changes]
..  include owis/TESTIS.HOLDATEN.F02
..  include owis/TESTIS.EXPORT.F02
..*****

```

Zur Individualisierung von Standard-Infosystemen siehe auch (*Individualisierung IS*).

Benennung INI-Dateien

Ablageort	Mandantenverzeichnis
Benennung	INI .XXXX oder XXXX .INI

Die Variablen, welche in INI-Dateien definiert werden sind mit dem Präfix "xy" anzulegen (siehe Benennung von Variablen).

Benennung von Variablen in FO/FO2 (.type/.var)

Variablen sollten generell sprechend sein. Eine kryptische Bezeichnung wie „x1“ ist nicht erlaubt. Alle benutzten Variablen müssen deklariert werden (ist durch die Interpreterzeile bereits vorgegeben).

Der generelle Aufbau einer Variable ist wie folgt:

[Präfix][Variablenart][Bezeichnung]

Zur Benennung von Variablen in Datenbanken/Infosystemen siehe (*Grundlagen: Benennung von Variablen*).

	.type/.var
Präfix	Als Präfix für allgemeine Variablen wird ein „x“ verwendet. Werden Parameter in INI-Dateien definiert, ist das Präfix ein „xy“. Werden Parameter in einem vorherigen Programm definiert und an ein Unterprogramm übergeben, so erhalten diese Programme das Präfix „xx“.
Variablenart	Orientiert sich an der Art der Variable. Für eine Textvariable (text) wäre die Variablenart ein „t“: t = Text, i = Integer, r = Real, b = Bool, d = Date, w = Week, p = Pointer bzw. Verweis, a = Aufzählung

	.type/.var
weitere Stellen	Der vollständige Variablenname muss sinnhaft zur Verwendung der Variable sein. Es ist besser, längere Variablenamen zu verwenden, weil sie besser gesucht werden können z.B. <code>xtselect</code> statt <code>xtsel</code>. Variablenamen sollen also wenn möglich nicht abgekürzt werden.
Nomenklatur	Variablen sind per CamelCase zu benennen, dadurch entsteht eine bessere Lesbarkeit: • <code>U xrAbrufMengeOffen</code> • <code>U xdHeute</code>

Events

Die Auflistung der Events wird benötigt für:

- Benennung von Programmen wie in den einzelnen Kapiteln zur Benennung beschrieben. Dort wird auf "Event" und die "Abkürzung EFOP" hingewiesen. Das Kürzel ist in der Spalte "Abkürzung EFOP" zu finden.
- Sprachneutrale Programmierung: In Spalte "Bezeichnung sprachunabhängig (G|evtartengl)" ist der Inhalt von G|evtartengl ersichtlich.

Ereignis	Bezeichnung (fop.txt)	Bezeichnung sprachunabhängig (G evtartengl)	Abkürzung EFOP
Maskeneintritt	maskein	SCREEN_ENTER	SE
Maskenprüfung	maskpruef	SCREEN_VALIDATION	SV
Workflow	workflow	SCREEN_WORKFLOW	WF
Maskenaustritt	maskaus	SCREEN_EXIT	SX
Maskenabbruch	maskabbr	SCREEN_CANCEL	SC
Multisite	spez. Event, wird generiert, s. Online-Hilfe		
Maskenende	maskende	SCREEN_END	SEE
Feld ausgefüllt	feldfuell	FIELD_FILL	FF
Feldprüfung	feldpruef	FIELD_VALIDATION	FV
Feldaustritt	feldaus	FIELD_EXIT	FX
Button vor	buttonvor	BUTTON_BEFORE	BB
Button nach	buttonnach	BUTTON_AFTER	BA
Zeile einfügen vor	zeileeeinvor	ROW_INSERTION_BEFORE	RIB
Zeile einfügen nach	zeileeeinnach	ROW_INSERTION_AFTER	RIA
Zeile löschen vor	zeileloevor	ROW_DELETION_BEFORE	RDB
Zeile löschen nach	zeileloenach	ROW_DELETION_AFTER	RDA
Zeile verschieben Prüfung	zeilebewpruef	ROW_MOVE_VALIDATION	RMV
Zeile verschieben vor	zeilebewvor	ROW_MOVED_BEFORE	RMB
Zeile verschieben nach	zeilebewnach	ROW_MOVED_AFTER	RMA
Zeile markieren	zeilemark	ROW_MARKED	RH
Zeile wechseln	zeilewechsel	ROW_CHANGED	RC
Berichtskopf	keine, da Infosystem-Feld		RPH
Berichtsfuss	keine, da Infosystem-Feld		RPF
Gruppenkopf	keine, da Infosystem-Feld		GRH

Ereignis	Bezeichnung (fop.txt)	Bezeichnung sprachunabhängig (G evtartengl)	Abkürzung EFOP
Gruppenfuß	keine, da Infosystem-Feld		GRF
Tabelle	keine, da Infosystem-Feld		TAB

Erläuterungen zu jeweiligen Events sind der Onlinehilfe zu entnehmen:
https://extranet.abas.de/sub_de/help/hd/html/9.1.11.2.html

Programmkopf und Interpreterzeile

Programmkopf

Um einen identischen Programmdurchlauf und Minimal-Basis für eine Dokumentation zu ermöglichen, sind folgende Header bei der Erstellung neuer FO- bzw. FO2-Programme zu verwenden. Diese können über die beigefügte XML-Datei in die Abas Tools als Template importiert und dadurch einfach verwendet werden.

Interpreterzeile im FO/FO2

Die erste Zeile eines FO/FO2-Programms muss die Interpreterzeile enthalten, in der die FO-Sprache anzugeben ist, in der programmiert wurde.

Hintergrund dieser Regel: Die FOP Sprache im Konfigurationssatz gibt an, ob die FOP-Kommandos deutsche oder englische verwenden. (Die Konfiguration findet sich in: Stammdaten > Firmenstamm > Anwendung > Konfiguration zeigen)

Über die Interpreterzeile in der Zeile 1 eines FOPs kann dies übersteuert werden. Wäre keine Interpreterzeile vorhanden, würde die in der Konfiguration hinterlegte Einstellung verwendet.

Beispiel: Ist in Konfiguration die FOP-Sprache Deutsch hinterlegt und die Interpreterzeile würde fehlen, dann würde das System deutsche FO-Befehle erwarten.

Die Interpreterzeile ist folgendermaßen anzugeben:

```
..!interpreter english noabbrev
```

Mögliche Optionen:

- **english/german** – Angabe, ob englische oder deutsche FO-Kommandos verwendet werden → Default: english
- **noabbrev** – Unterbindet das Abkürzen von Variablen, dies ist eine Pflichtangabe. Die Option "noabbrev" impliziert die Option "declaration", d.h. Variablen müssen vor dem Verwenden deklariert werden. Die Angabe "declaration" ist daher nicht nötig.

Optionen, die nicht verwendet werden dürfen:

- **englvar** – Gibt an, dass sämtliche Variablen mit englischem Variablennamen angesprochen werden müssen. Diese Option darf **nicht** verwendet werden (Erklärung siehe Sprache).
- **translate** – Ist die Option translate gesetzt, wird bei einer Übersetzungsanforderung die Befehlssprache dieses Programms umgestellt. Diese Option darf **nicht** verwendet werden, weil eine solche Übersetzung in der Regel nicht gewünscht ist.

.set trans - / .set trans +

Seit der Version 2100r0 werden FOPs standardmäßig zur Laufzeit übersetzt. Wird ein FOP mit **.input** eingezogen, gilt dort standardmäßig **.set trans +**.

Das gesamte FOP wird also übersetzt, was zu Problemen in fremdsprachlichen Umgebungen führen kann.

Generell muss daher in jedem FOP nach der Interpreterzeile ein `.set trans -` eingefügt werden, um dies zu verhindern. Sollen Abschnitte übersetzt werden, so muss dieser Abschnitt mit `.set trans +` beginnen und mit `.set trans -` wieder beendet werden um die Übersetzung zu aktivieren. Texte aus einigen FO-Kommandos (`.error`, `.box`, ...) werden immer vom Kern übersetzt. Siehe auch <https://innosearch.abas.de/inno/de/ShowInno.html#query=nummer:20200135>.

Grundsätzlich gilt, dass in Projekten, in denen es Umgebungen mit mehreren Bediensprachen gibt, festgelegt werden muss, wie mit fremdsprachlichen Übersetzungen umgegangen wird.

Sprachunabhängige Programmierung

Generell ist darauf zu achten, dass FO-Programme sprachunabhängig programmiert sind. Das Ergebnis eines Programms darf nicht variieren, nur weil der Benutzer sich in einer anderen Sprache anmeldet.

(Ausnahme sind die sprachabhängigen Übersetzungen für Meldungen und Abfragen)

Zum Thema Mehrsprachige Programmierung existiert auch ein Beitrag in der Online-Hilfe:

https://extranet.abas.de/sub_de/help/hd/html/1.2.4.html

Eventkommandos und Eventtaste sprachneutral

Die Eventkommandos "neu", "zeigen", "ändern" etc. müssen sprachunabhängig abgebildet werden.

Durch einlesen des Standardprogramms "EVT_CMD.CONST" stehen die Eventkommandos sprachneutral zur Verfügung und können damit abgefragt werden.

Die Ausführungen hier gelten auch für G|avtkommd und G|avttaste.

G|evtkommd (bzw. G|avtkommd)

Beispiel: Eventkommando „neu“

Falsch: G|evtkommd = "neu"

Richtig:

```
.input EVTCMD.CONST
if (G|evtkommd = U|evtcmdNew) {
  ..
}
```

G|evttaste (bzw. G|avttaste) beim Maskenende

Auch G|evttaste (bei Ereignis Maskenende muss dies geprüft werden) kann sprachneutral angegeben werden:

Falsch: G|evttaste = "fertig"

Richtig:

```
.input EVTCMD.CONST
if ((G|evtkommd = U|evtcmdNew ; G|evtkommd = U|evtcmdEdit) & G|evttaste = U|evtcmdDone) {
  ..
}
```

Art des Ereignisses sprachneutral (G|evtartengl)

Die Art des Ereignisses ist ebenfalls sprachunabhängig anzugeben.

Beispiel: `G|evtartengl = "FIELD_EXIT"`

Hinweis: Es gibt z.B. diese Möglichkeiten, um den Inhalt von G|evtartengl herauszufinden:

- siehe Tabelle Events in diesem Dokument
- In einer Maske rechte Maustaste → FOP → Trace für alle Ereignisse und das entsprechende Ereignis auslösen. Im Textfenster steht dann z.B. "Event : SCREEN_VALIDATION (maskpruef)" und damit weiß man, dass es SCREEN_VALIDATION heißt, wenn man in der Maskenprüfung ist.

Datenbankangabe in Englisch (z.B. .select/.load)

Durch die FO-Sprache Englisch müssen die Datenbanken auch in Englisch angegeben werden (z.B. bei Verwendung von .select bzw. .load)

Richtig: `.load 0 nolock sales 'U|xtSelect'`

Falsch – Sprachunabhängig via Array G|hdatname: `.load 0 'G|hdatname3' nolock 'U|xtSelect'`

→ Die Nutzung der Angabe mit Array darf **nicht** genutzt werden, weil die Programme damit schlechter lesbar werden.

Möglichkeiten zur Ermittlung des englischen Datenbanknamens

Standardprogramm s3sys/KOMLIST

Das Standardprogramm s3sys/KOMLIST generiert eine Ausgabe mit deutschen sowie englischen Begrifflichkeiten und ist in jeder Installation vorhanden.

Ausführung per Expertenmodus: `<(Text)> s3sys/KOMLIST <(View)>`

Für den Datenbanknamen Menüpunkt "Dateinamen" wählen. Der englische Datenbankname steht in der Liste in der Spalte "Dateiname (.hol)".

Variablentabelle: Datenbanknummer und Anwendung hdatname.x zur Ermittlung englischer Dateiname

Die Variablentabelle zum Datensatz zeigt verschiedene Informationen an.

Button "Zusätzliche Informationen" → Register Details zeigt das Feld "Schlüsselwort für FOP" in der Sprache der FO-Sprache der Konfiguration.

Wäre in der Konfiguration die FO-Sprache Englisch eingestellt, würde dann hier z.B. für V-01-01 (Lieferantenstamm) "supplier" stehen. (bei deutscher FO-Sprache: lieferant)

In der Variablentabelle gibt es auf Register Allgemeines auch die Angabe des Datenbanknamens. Die englische Bezeichnung in "Datenbankname für Kommandos" (vdntxts) kann aber von der FO-Bezeichnung abweichen! (z.B. V-01-01 für Lieferantenstamm heißt "(Vendor)").

Wenn also die FO-Sprache Deutsch ist, könnte man sich die englische FO-Bezeichnung über die Datenbanknummer und der Variable hdatname ermitteln.

In der Variablentabelle steht auf Register "Allgemeines" die Datenbanknummer. Für den Lieferantenstamm V-01-01 ist das die Nummer 1.

z.B. im Direktmodus eingeben:

```
..!interpreter english  
'hdatname1'
```

→ damit erhält man die Ausgabe "supplier" und kann diese Angabe nun im eigenen .select/.load verwenden.

Beispiele für wichtige Datenbanken

Datenbank	Datenbanknummer	FOP-Datenbankname in Englisch
Kunde	0	customer
Lieferant	1	supplier
Artikel	2	parts
Verkauf	3	sales
Einkauf	4	purchasing
Betriebsauftrag	9	workorder
Zusatzdatenbank 1	15	file1
Zusatzdatenbank 2	18	file2
Lagermenge	40	quantity

Zur Nutzung von **.select**, **.load** bzw. **.add** muss dieses Kapitel beachtet werden: (Kommandos Datenbankzugriffe).

Kommandosyntax

Falls bspw. per **.command** ein Kommando per FO-Programm ausgeführt werden soll, ist dieses ebenfalls sprachunabhängig anzugeben, dies betrifft Datenbankname, Modus und Gruppe.

Richtig: **<(Customer)> 4711 <(View)>**

Falsch: **<Kunde> 4711 <Zeigen>**

Dateinamen, Dateiaktionen, Tippkommandos, Sonstige (Laderkommandos) (s3sys/KOMLIST)

Das Standardprogramm s3sys/KOMLIST generiert eine Ausgabe mit deutschen sowie englischen Begrifflichkeiten und ist in jeder Installation vorhanden.

Ausführung per Expertenmodus: `<(Text)> s3sys/KOMLIST <(View)>`

und Auswahl der gewünschten Inhalte. Ausgewählt werden können:

- Dateinamen
- Dateiaktionen
- Tippkommandos

Aufzählungen

Es ist generell der sprachunabhängige Wert zu verwenden.

Falsch: `'M|vorgartaz'` Ergebnis: LFABRUF

Richtig: `'M|vorgartaz(L=":")'` Ergebnis: (ForecastDeliverySchedule)

Wird ein Aufzählungsfeld in einer Bedingung, Zuweisung oder in Selektionsstrings genutzt, dann ist es entsprechend anzugeben.

Richtig:

Bedingung:

```
.continue VARIANT ? 'H|dispoa(L=":")'="(VariantRelated)"
```

Zuweisung:

```
.formula H|dispoa = "(VariantRelated)"
```

Selektionsstring:

```
.select parts "$,,dispoa=(VariantRelated);@ablageart=(Active)"
```

Der Inhalt für die sprachneutrale Angabe findet man entweder in der Maske Aufzählung oder wie folgt:

Bei Standard-Aufzählungen findet man die sprachneutralen Möglichkeiten so:

- Manuelle dynamische Selektion mit betreffendem Feld öffnen und Selektionskriterien anschauen. z.B. benötigt man für das Feld Dispositionsart den sprachneutralen Inhalt für "variantenbezogen".
Dynamische Selektion öffnen mit `%,dispoa=variantenbezogen`. In Objektauswahl → Kommando → zur Kommandoübersicht hinzufügen (mit Selektionskriterien) aufrufen. Das sieht dann z.B. so aus:
`<(Part)> %,dispoa==(VariantRelated);@gruppe=1;@ablageart=(Active) <(View)>`
und man sieht den sprachneutralen Begriff.
- `edpexport.sh -a AP -l (Art der Aufzählung)` – z.B. benötigt man für das Feld Dispositionsart den sprachneutralen Inhalt. Feldart ist A131. Im Expertenmodus: `!edpexport.sh -a AP -l A131 -p (passwort)`. Infosystem SYSTEM: `edpexport.sh -a AP -l A131 -P` (und

"Einmalpasswort" anhaken).

Einheiten

Auslesen verschiedener Informationen eines Einheitendatensatzes.

Beispiel über Abstieg in TAZ1 Feld <le> (Lagereinheit) im VK Auftrag:

Punkt	Kommando	Rückgabewert	Bezeichnung
1	'F value(M le,"TAZ1")'	kg	Bezeichnung der Einheit
2	'F value(M le,"TAZI1")'	KGM	Bezeichnung der Einheit nach ISO 7372
3	'F value(M le,"TAZS1")'	(KGM)	Bezeichner für Programme
4	'F value(M le,"TAZU1")'	(166,62,0)	Datensatz-ID

Übersetzungen Wörterbuch

Es gibt zwei verschiedene Arten von Texten, welche mittels des Wörterbuchs übersetzt werden müssen:

- Texte zur Bedienbarkeit (Bediensprache)
- Texte zur Kommunikation nach außen (Korrespondenzsprache)

Die aktivierten Bedien- sowie Korrespondenzsprachen werden im Konfigurationsdatensatz eingestellt (Stammdaten > Firmenstamm > Anwendung > Konfiguration zeigen).

Übersetzung der Bediensprache: Wird in einem Programm eine Bildschirmausgabe generiert, bspw. über eine Box, oder ein Text für eine Druckausgabe erstellt, so muss dieser zur Übersetzung des ausgegebenen Textes in das Wörterbuch aufgenommen werden.

Zu übersetzende Texte können mit 2 Methoden in das Wörterbuch aufgenommen werden:

1. Kommando **.set trans**

```
.set trans +  
.formula U|xtext = "Lieferschein"  
.set trans -
```

2. Kommando **.text**

```
.text U|xta 'P|spr'   Lieferschein
```

Batch-User (für EDP-Aufrufe)

Um Rechte-Probleme bei der Ausführung der EDP-/EPI-Skripten zu vermeiden, muss im Betriebsdatensatz von jedem Mandanten ein Verweis auf einen Passwort-Datensatz angelegt werden. In diesem wird der Batch-User hinterlegt. Dies sorgt dafür, dass nur befugte Personen den Batch-User einsehen und ändern können.

Dieses Passwort soll für alle EDP-/EPI-Ausführungen verwendet werden, sofern diese dadurch keine Lizenz einschränkungen umgehen. In der Passwortdefinition des Batch-Users muss die Option "fremd nutzbar" aktiviert sein.

Beispiel:

```
.assign G|neuespassw = F|sysinfo(10)^ybatchpw^id  
.system „epi....“ password background
```

Alternativ kann der Batch-User auch in einer INI Datei definiert werden. Dies ist projektabhängig festzulegen. Auch in diesem Fall ist sicherzustellen, dass der Batch-User nicht von unbefugten Personen eingesehen oder geändert werden kann.

Dokumentation der Anpassungen

Die Dokumentation der Anpassungen ist essentiell für die Wartbarkeit und das Verständnis der einzelnen Programme. Im Folgenden wird vertieft, wie die Dokumentation im Programm durchzuführen ist (sog. Inline-Doku).

An einer Stelle im Programmkopf muss ein Verweis auf ein Arbeitspaket, Ticket o.ä. erfolgen, um darüber an eine Aufgabenbeschreibung gelangen zu können. Ziel ist es, dass ein Außenstehender mit diesen Angaben in kurzer Zeit die Lösung verstehen und ggf. erweitern oder auf Fehler prüfen kann.

Dokumentation von Abschnitten (FO) bzw. Prozeduren (FO2)

Dokumentation im Quellcode

Um den Quellcode nachvollziehbarer zu machen, sind ebenfalls einzelne Zeilen zu dokumentieren, die nicht auf den ersten Blick verständlich sind.

```
..
```

Es soll nicht jede Programmzeile kommentiert werden, nach Möglichkeit sind die Grundsätze von Clean Code einzuhalten, so dass Programmcode möglichst gut lesbar ist und ohne Kommentare verstanden werden kann. Für die Programnteile, in denen dies nicht möglich ist, sind Kommentare jedoch erforderlich.

Es sind Metatags für alle verwendeten Puffer anzugeben, da damit sowohl die Autovervollständigung in den Abas Tools funktioniert, aber auch die Nachvollziehbarkeit von Programmcode erhöht wird.

Sprachenauswahl für die Programmdokumentation

Die Sprache zur Dokumentation im Sourcecode ist: **Englisch**

Bestehende Anpassungen erweitern

Generell gilt: Eine Anpassung wird in der Programmiersprache erweitert, in der diese auch erstellt wurde. Sollten Änderungen vorgenommen werden, müssen diese auch wie folgt dokumentiert werden:

[Firmenname]-[VNachname] [Tagesdatum] [Grund der Änderung]

Beispiel: **abas-MMustermann 11.11.2026 AP4711: approval of work order suggestions**

Sollten Funktionalitäten abgeschaltet werden, so werden diese keinesfalls gelöscht! Die nicht mehr benötigten Zeilen werden wie Änderungen dokumentiert und auskommentiert. Ungenutzter Programmcode wird entfernt, siehe auch (*Grundlagen: Ungenutzter Code*).

FO-Kommandos – Besonderheiten

Allgemeines (Puffer, keine Abkürzung)

Bei jedem Zugriff auf eine Variable ist der Puffer, aus dem die Variable verwendet wird zwingend anzugeben. Dies dient der besseren Lesbarkeit.

Eine Übersicht der Puffer ist der Onlinehilfe zu entnehmen:
https://extranet.abas.de/sub_de/help/hd/html/9.1.4.6.html

Kommandos und Optionen dürfen nicht abgekürzt werden.

Formatierung

Es ist auf saubere Formatierung zu achten, die die Lesbarkeit verbessert. Das schließt Leerzeichen, ggf. Tabstopps und Klammerungen ein. In den AbasTools kann mit der Tastenkombination **Strg + Shift + F** das gesamte Programm automatisch formatiert und eingerückt werden.

Temporärdateien

Es ist darauf zu achten, dass die Lebensdauer von Temporärdateien nur solange ist, wie sie benötigt werden. Nach erfolgreicher Verarbeitung müssen diese gelöscht werden. Generell sollten temporäre Dateien im Verzeichnis **rmtmp** abgelegt werden. Dateien, die nur zur Laufzeit benötigt werden, sollten mit **.file -tempname** generiert werden. Es ist sicherzustellen, dass temporäre Dateien, die nicht auf diese Art angelegt wurden, nach der Verarbeitung gelöscht werden.

Siehe auch https://extranet.abas.de/sub_de/documentation/de/online-help-page/online-help.html?target=%2Fsub_de%2Fhelp%2Fhd%2Fhtml%2F12.36.1.html%2312.36.1.3&lang=de#german-oh

Protokollierung

Für jede Hintergrundverarbeitung wie bspw. Cronjobs ist eine Protokollierung vorzunehmen. Es ist pro Thema bzw. aufrufendem Programm eine separate Protokolldatei zu schreiben. Diese muss maschinenlesbar sein, d.h. jeder Eintrag muss dem gleichen Aufbau folgen mit identischen Trennzeichen, so dass bspw. eine einfache Auswertung per Excel möglich ist.

Protokolle sind im Verzeichnis **owlog** abzulegen. Es ist darauf zu achten, dass nicht unnötig viele kleine Protokolldateien entstehen. Es ist ebenfalls darauf zu achten, dass Protokolldateien nicht unnötig groß werden.

Fehlerdateien

Bei EDP-/EPI-Ausführungen gehört zur Fehlerdatei auch immer die passende Eingabedatei, da der Fehler sonst nicht nachvollzogen werden kann. Diese Dateien müssen entweder per Dateiname oder Ablage in einem Unterverzeichnis klar zuordenbar sein. Die Speicherung muss persistent erfolgen, nach Bereinigung eines Fehlers müssen diese dann gelöscht werden.

Kommandos für Datenbankzugriffe: **.select** bzw. **.add** bzw. **.load**

Nach einem solchen Kommando, welches auf die Datenbank zugreift ist zwingend auf **G|mehr** zu prüfen, um bei Nichterfolg der Aktion kontrolliert fortzufahren.

Ausnahme: **.select application** setzt nicht **G|mehr**.

Es sollte sich immer bewusst gemacht werden, welche Sperroption benötigt wird und diese auch explizit angegeben werden. Insbesondere bei Lesevorgängen im Einkauf und Verkauf und Fertigung ist zu prüfen, ob der Default (**readlock**) erwünscht ist. In den meisten Fällen ist die Angabe von **no lock** sinnvoll, um den Vorgang bei langlaufenden Programmen nicht für andere Zugriffe unnötig zu sperren.

Erklärung zur Lesesperre bei **.select** bzw. **.load** ohne Sperroption

Wird ein Datensatz mit dem Kommando **.select** bzw. dem Kommando **.load** gelesen, kommt es auf das Datenbankobjekt an, ob ein Datensatz mit Sperre oder ohne Sperre gelesen wird.

Folgende Datenbankobjekte lesen automatisch mit Lesesperre:

- Einkauf
- Verkauf
- Fertigung

Wenn ein Verkaufs-/Einkaufs- oder Fertigungsverfahren ohne **no lock** gelesen wird, dann wird eine Sperre auf den Vorgangskopf gesetzt, so dass niemand den Kopf oder eine dazu gehörende Zeile ändern kann. Während das FOP die Zeilen durchläuft, sieht man immer den Zustand zum Lesezeitpunkt des Kopfs, die Daten sind also konsistent.

Liest man einen Verkaufs-/Einkaufs- oder Fertigungsverfahren dagegen mit **no lock**, dann kann es sein, dass während des Zeilendurchlaufs eine Zeile durch einen Benutzer geändert wird. Diese hat dann einen anderen Zustand als zum Lesezeitpunkt des Kopfes. Das kann dazu führen, dass das FOP ggf. falsche Daten berechnet.

Beispiel des Verhaltens für den Anwender: Ein FOP liest einen Verkaufsdatensatz mit dem Kommando **.select sales "200189"** und erzeugt eine Lesesperre. Zur gleichen Zeit wird von Benutzer B der Auftrag 200189 im Modus Ändern geöffnet und der Benutzer nimmt Änderungen vor. Speichert Benutzer B den Auftrag und das FOP steht immer noch auf dem Datensatz, erscheint für Benutzer B die Meldung "Datensatz belegt" mit dem Hinweis "ACHTUNG! Abbrechen führt zum Abbruch der Anwendung und zum Verlust aller Eingaben." Das führt zum Abbruch der Anwendung, d.h. die Maske Auftrag wird ohne Speichern sofort geschlossen. Man kann die Maske also nur speichern, indem man auf die Freigabe der Sperre des FOPs wartet.

Angabe Datenbankname beim Selektieren

Beim Selektieren mit FOP muss der Datenbankname in der Selektion angegeben werden. Dies erhöht die Lesbarkeit von Programmen.

Nicht erlaubt ist z.B.:

- `.select/.load continue`
- Selektionsstring mit Angabe der Datenbank darf nicht verwendet werden (`@database=x`)
- `.select/.load nolock 0:2`

Ausnahme für `.select/.load object`: Die Selektion mit "object" darf nur für das Lesen einer einzelnen ID genutzt werden. Erlaubt ist also: `.load 0 nolock object 'M|id'`

Kommando zum Zurückschreiben bzw. Erstellen von Datensätzen: `.rewrite`

Falls in einer Installation Multisite zum Einsatz kommt, ist in den zum Austausch konfigurierten Datenbanken/-gruppen die Nutzung von `.rewrite` verboten, da dies Fehler verursacht.

Auch ohne Multisite ist das Kommando zu vermeiden, weil bei `.select lock / .rewrite` keine EFOPs ausgelöst werden. Besser ist die Nutzung von EDP/EPI bzw. AJO.

Wird bei der Änderung aus bestimmten Gründen die Ausführung ohne EFOPs benötigt, dann kann im EDP folgendermaßen vorgegangen werden:

- über das SET den FOPMODE ausschalten.
- das Kommando MODIFY nutzen. Damit werden auch keine EFOPs ausgeführt und es gelten die Regeln der Beschreibbarkeit der Felder wie bei `.select lock`.

Unterprogramme

Bei der Nutzung von Unterprogrammen sind diese standardmäßig mit `.continue` zu beenden und nur im bewussten Fehler-/Abbruchfall mit `.end X`, damit das aufrufende Programm im Anschluss weiterlaufen kann.

Bei Unterprogrammen, die dazu dienen bestimmte Eigenschaften zu setzen (bspw. DATEI.F zum Setzen der Parameter für eine Dateiausgabe), sollte darauf geachtet werden, dass diese nicht unnötig mehrfach eingezogen werden.

Transaktionen

Transaktionen dienen zur Synchronisierung von Schreibzugriffen auf die Datenbank. Dies ist insbesondere wichtig und relevant, wenn einmalige Aktionen (wie bspw. die Buchung von Lieferscheinen oder Rechnungen) und andere Aktionen miteinander kombiniert werden und sichergestellt werden muss, dass alle Aktionen auch fehlerfrei ausgeführt wurden, um im Fehlerfall keinen inkonsistenten Datenbestand zu hinterlassen.

Hierbei werden jedoch Sperren in der Datenbank gesetzt und gehalten und es werden Daten vorgehalten, um zurückrollen zu können. Dies beeinträchtigt andere Prozesse und verlangsamt das System, so dass Transaktionen immer nur ganz bewusst eingesetzt werden dürfen, also nur für eine unbedingt zusammenhängende Datenänderung. Diese sollte so klein wie möglich gehalten werden.

Siehe auch: https://extranet.abas.de/sub_de/help/hd/html/FO.AKTION.html

Eine Transaktion ist nur dann wirksam, wenn innerhalb dieser nicht zusätzliche Anmeldungen erfolgen (wie bspw. eine neue Anmeldung per EDP).

Bei der Verwendung von Transaktionen bei Lagerbuchungen sollte man auch berücksichtigen, dass ggf. eine Sperre auf den Nummernzähler PSLITEM gesetzt und gehalten wird.

Programmausgaben bzw. Interaktion (**.box**, **.menu**, **.lesen**, etc.)

Jegliche Programmausgaben bzw. Interaktion müssen das Thema Hintergrundverarbeitung per EDP/EPI oder Multisite berücksichtigen.

Für Menüausgaben müssen Default-Werte definiert werden, welche bei einer Hintergrundverarbeitung verwendet werden, die Prüfung sollte hier auf **G|edp** erfolgen. Falls hier eine Unterscheidung bei Multisite erforderlich ist, muss dies per **P|multisiteimport** geprüft werden.

Webclient	G batchmode = G false & G grafik = G false
GUI	G batchmode = G false & G grafik = G true
EDP	G batchmode = G true & G grafik = G false

Schutz von Feldern: **.protection**

Sofern ein per **.protection** geschütztes Feld per EDP/EPI oder Multisite geschrieben werden soll, führt dies zu einer Fehlermeldung. Daher darf **.protection** nur verwendet werden, wenn gleichzeitig auf **G|edp** geprüft wird.

Richtlinien – speziell FO

Programmkopf

FO-Header

Version für Template:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<templates>
<template autoinsert="false" context="de.abas.tools.fop.Fop.TagLine" deleted="false"
description="FO header and body template" enabled="true" id="de.abas.tools.fop.Fop.HeaderAndBody"
name="FO header and body new">
..!interpreter english declaration noabbrev
.. *****
.. Author          : [Author]
.. Date of Creation : ${date}
.. Name            : ${filename}
.. Work package    :
.. Function        :
.. Modification     : [Author] [Date] [Changes]
..                 [Author] [Date] [Changes]
.. *****
..&#13;
..&lt;BLOCK&gt; Verwendete Puffer&#13;
..&lt;META P|='P88:10' &gt;&#13;
..&lt;/BLOCK&gt;&#13;
..
</template>
</templates>
```

Version für direkte Kopie:

```
..!interpreter english declaration noabbrev
.. *****
.. Author          : [Author]
.. Date of Creation : ${date}
.. Name            : ${filename}
.. Work package    :
.. Function        :
.. Modification     : [Author] [Date] [Changes]
..                 [Author] [Date] [Changes]
.. *****
..set trans -
..<BLOCK> Verwendete Puffer
..<META P|='P88:10' >
..</BLOCK>
..
```

Bestandteile

Sprungmarken

Bei Sprungmarken werden lediglich die ersten 7 Zeichen ausgewertet, daher sollten aus Gründen der Übersichtlichkeit auch maximal 7 Zeichen genutzt werden, um nicht durch Änderungen im hinteren Teil mögliche Fehler durch (für den Interpreter) identische Sprungmarken zu provozieren. Wenn es nicht bereits durch ein Block-Statement dokumentiert wurde, kann hinter einer Sprungmarke mit einem Doppelpunkt abgetrennt ein Kommentar hinterlegt werden.

Richtlinien – speziell FO2

Programmkopf

FO2-Header

Version für Template:

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<templates>
<template autoinsert="false" context="de.abas.tools.fop.Fop.TagLine" deleted="false"
description="FO2 header and body template" enabled="true" id="de.abas.tools.fop.Fop.HeaderAndBody"
name="header and body new">
..!interpreter english declaration noabbrev
.. *****
.. Author          : [Author]
.. Date of Creation : ${date}
.. Name            : ${filename}
.. Work package    :
.. Function         :
.. Modification     : [Author] [Date] [Changes]
..                 [Author] [Date] [Changes]
.. *****
..&#13;
..&lt;BLOCK&gt; Verwendete Puffer&#13;
..&lt;META P|='P88:10' &gt;&#13;
..&lt;/BLOCK&gt;&#13;
..
..&lt;VAR folded&gt;
..&lt;/VAR&gt;

..&lt;BLOCK&gt; Main Methode def main(){ } ..&lt;/BLOCK&gt;

</template>
</templates>
```

Version für direkte Kopie:

```
..!interpreter english declaration noabbrev
.. *****
.. Author          : [Author]
.. Date of Creation : ${date}
.. Name            : ${filename}
.. Work package    :
.. Function         :
.. Modification     : [Author] [Date] [Changes]
..                 [Author] [Date] [Changes]
.. *****
..set trans -
..<BLOCK> Verwendete Puffer
```

```
..<META P|='P88:10' >  
..</BLOCK>  
..  
..<VAR folded>  
..</VAR>  
  
..<BLOCK> Main Methode def main() { } ..</BLOCK>
```

Bestandteile

Main Methode

Die Main Methode stellt den zentralen Programmablauf dar. Hier sollte nur auf Einstiegskriterien für den weiteren Programmverlauf geprüft werden und weitere Prozeduraufrufe erfolgen.

Beispiel:

```
..<BLOCK>  
def main() {  
  .input EVTCMD.CONST  
  .. Only execute in mode "new"  
  if (G|evtkommd = U|evtcmdNew) {  
    .. Only execute if customer is filled  
    if (_F|empty(M|kunde)) {  
      .call getConsultant()  
    }  
  }  
}
```

Prozeduren

Innerhalb der Prozeduren wird die Verarbeitungslogik erstellt, d.h. Datenbeschaffung, Datenmanipulation, etc. Eine Prozedur sollte immer eine klare Funktion erfüllen und nicht mehrere verschiedene Funktionalitäten abdecken.

Individualisierung von Infosystemen (IS)

Falls es sich bei einem IS um ein AJO-IS handelt, sind die folgenden Punkte in der Online-Hilfe zu beachten:
https://extranet.abas.de/sub_de/help/hd/html/66.44.html

Bei reinen FO-IS gelten folgende Regeln:

- Das Infosystem-Objekt muss kopiert werden und erhält damit eine neue Identnummer (mindestens 6-stellig).
 - Als Arbeitsbereich wird dem ursprünglichen Arbeitsbereich ein "ow" vorangestellt (z.B. vk → owvk).
 - Das Suchwort bleibt unverändert, damit sich das individuelle Infosystem auch aus dem Standard Register der Kommandoübersicht öffnen lässt. Druck-Infosysteme werden mit dem Präfix "I" versehen.
 - Bei der Kopie von Infosystemen kann das Infosystem FOPCOPY unterstützen: [Online-Hilfe](#).
- Es müssen alle FOPs des Infosystems individualisiert werden. Hierzu werden diese ebenfalls in die jeweiligen ow-Verzeichnisse kopiert.
 - Dies gilt für alle Ebenen, d.h. auch für Sub-FOPs und deren Sub-FOPs.
 - Die Aufrufe dieser Sub-FOPs sind entsprechend zu tauschen, so dass die individuellen FOPs aufgerufen werden.
 - Zusätzlich sind alle Sub-FOPs per **include** im Kopf der entsprechenden Programme zu dokumentieren bzw. vorhandene **include**-Zeilen auf die neu eingefügten FOPs anzupassen, so dass der IS-Export die korrekten FOPs vollständig mit exportieren kann für eine spätere Übertragung auf andere Mandanten.
 - Dies gilt nicht für FOPs aus dem Arbeitsbereich **is**, diese bleiben im Standard.
- Zudem müssen alle Aufrufparameter geprüft werden:
 - Aufrufparameter, die in dem Standard-IS angebunden sind und andere Masken, IS oder Aktionen ausführen sind zu kopieren und an dem neuen IS anzubinden.
 - Aufrufparameter, die das Standard-IS aus anderen Masken heraus aufrufen, sind zu kopieren und an die jeweiligen anderen Masken anzubinden. Die Standard-Aufrufparameter werden deaktiviert, so dass nur noch der Aufruf auf das individuelle IS vorhanden ist. Dies stellt den Standardfall dar, bei dem das individualisierte IS das Standard-IS ersetzt. Falls dieses parallel dazu genutzt werden soll, kann hiervon abgewichen werden.
 - Bei der Kopie der Aufrufparameter kann das Standard-Infosystem APLADMIN unterstützen (siehe [Online-Hilfe](#)).
- Es sind alle an das Standard-IS angebundenen Drucklayouts und deren Struktur und zugehörigen Objekte zu kopieren und an das neue IS anzuhängen.

Druckindividualisierung

- Grundsätzliche Regel bei den Umsetzungen der Anpassungen: Es soll möglichst alles an Anpassungen hinsichtlich Datenmanipulation per FOP umgesetzt werden und so wenig wie möglich bzw. am besten gar keine Logik in den Jasper-Layouts selbst verwendet werden.
- Ziel sollte es sein, möglichst alle Anpassungen in den jeweiligen i-Files (z.B. **FOPKOPF**, **FOPZEILE**, **FOPENDE** etc.) des Infosystems vorzunehmen. FO2-Programme sollten in diesen i-Files eingezogen werden und nicht in den Hauptprogrammen (wie **EVVORGANG . TAB**). Die Benennung der FOPs soll dabei einheitlich erfolgen, z.B. **XFOPENDE . ANPASSUNGEN . FO2**.
- Wenn eine Anpassung in regulären Datenbeschaffungsprogrammen wie **EVVORGANG . TAB** oder **DRUCK . BN** unumgänglich sein sollte, so ist diese innerhalb des Programms in FO durchzuführen und sowohl bei der betreffenden Änderung als auch im Programmkopf klar und deutlich (s. dieses Dokument: Bestehende Anpassungen erweitern) zu markieren und dokumentieren.
- Bei Bestandsinstallationen ist in der bestehenden Programmiersprache weiterzuentwickeln. Das Einbinden von FO2-Subprogrammen in bereits angepassten FOs ist nicht zulässig. Sind die Anpassungen noch überschaubar, kann das i-File jedoch vollständig in FO2 überführt werden.
- Pro i-File sollte nur ein korrespondierendes FO2 vorhanden sein. Weitere funktionale Unterteilungen sind darin per Prozeduren abzubilden.
- Sollte das Löschen von Zeilen notwendig sein, sollte dies möglichst als Nachläufer im ENDE-FOP (z.B. **XFOPENDE . ANPASSUNGEN . FO2**) erfolgen.
 - Es empfiehlt sich gerade im **FOPENDE** eine Prozedur zum Löschen von Zeilen einzufügen, die jeweils aufgerufen werden kann. Gegebenenfalls kann diese gleich nach der Individualisierung angelegt werden.
- Aufgrund der Sprachabhängigkeiten ist mit **.text** und ***.P|ksi** zu arbeiten, z.B. **H|name.P|ksi**. Beim Auslesen von Datums- und Real-Feldern in Textfelder ist auf die entsprechende Abhängigkeit von der Korrespondenzsprache zu achten (**L=P | spr**), um die in der Konfiguration möglichen Formatierungen automatisch zu berücksichtigen.
- Empfehlungen Anpassungen im Jasper-Report:
 - Keine Verwendung von leeren Gruppen/Bändern
 - Möglichst keine Leerräume am Ende eines Bandes
 - Möglichst keine zusammengesetzten Felder verwenden
 - Freitexte möglichst in einem eigenen Band verwenden
 - Bilder müssen in einem Pfad relativ zum win-Verzeichnis liegen oder eine absolute, vom System auswertbare Pfadangabe haben, damit sie von der Jasper-Reports-Engine gefunden werden können.
 - Die Verwendung von statischen Texten im Layout ist nur in Ausnahmefällen zulässig (z.B. rein internes Dokument mit viel Text aber wenig dynamischen Inhalten aus Abas ERP, nur auf expliziten Kundenwunsch), in allen anderen Fällen sind Variablen/Bezeichner des Druckinfosystems zu verwenden. Alternativ können Ressource Bundles verwendet werden, um größere Mengen statischen Texts abzubilden (siehe z.B. im Standard: Saldenbestätigung, Versandplanung).
 - Bei der dynamischen Formatierung von Feldern (Farbe/fett/kursiv/...) in Abhängigkeit von Datenkonstellationen sind conditional styles zu verwenden, die Duplizierung von Feldern ist zu unterlassen.
 - Das Duplizieren ganzer Layoutstrukturen (z.B. TABELLE/ZEILEN) ist nur in Ausnahmefällen zulässig

(z.B. bei Erstellung eines gänzlich neuen, individuellen Vorgangstypen).

- Die JRXML-Dateien sollten grundsätzlich lokal bearbeitet werden, damit die Version auf dem Kundensystem nicht versehentlich beschädigt oder überschrieben wird.
- Hinweis: Einige dieser Empfehlungen verringern das Problem des fehlerhaften Übertrags.